

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



Sorting

Oleh:

Amalia Soraya

NIM. 2510817120002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4 : Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya
NIM : 2510817120002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
BUBBLE SORT	1
A. Source Code.....	1
B. Output Program	2
C. Pembahasan	2
SELECTION SORT	4
A. Source Code.....	4
B. Output Program	5
C. Pembahasan	5
INSERTION SORT	7
A. Source Code.....	7
B. Output Program	8
C. Pembahasan	8
MERGE SORT	10
A. Source Code.....	10
B. Output Program	12
C. Pembahasan	12
QUICK SORT	14
A. Source Code.....	14
B. Output Program	15
C. Pembahasan	15

RADIX SORT	17
A. Source Code.....	17
B. Output Program	19
C. Pembahasan	19
ANALISIS PERFORMA	20
TAUTAN GIT	21

DAFTAR GAMBAR

Gambar 1. Screenshot Output Bubble Sort	2
Gambar 2. Screenshot Output Selection Sort	5
Gambar 3. Screenshot Output Insertion Sort.....	8
Gambar 4. Screenshot Output Merge Sort.....	12
Gambar 5. Screenshot Output Quick Sort	15
Gambar 6. Screenshot Output Radix Sort	19

DAFTAR TABEL

Tabel 1. Source Code Bubble Sort	1
Tabel 2. Source Code Selection Sort.....	4
Tabel 3. Source Code Insertion Sort.....	7
Tabel 4. Source Code Merge Sort	10
Tabel 5. Source Code Quick Sort	14
Tabel 6. Source Code Radix Sort	17

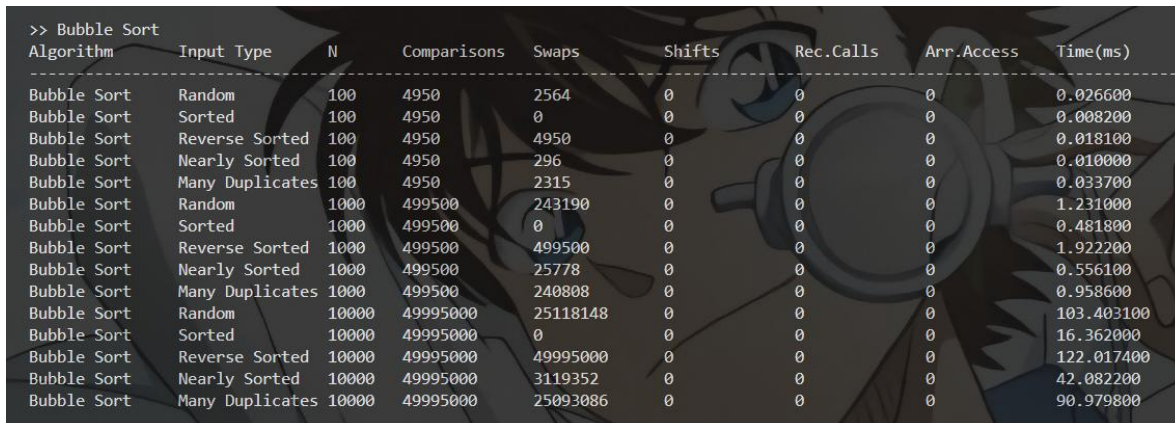
BUBBLE SORT

A. Source Code

Tabel 1. Source Code Bubble Sort

1	<code>void bubble_sort(std::vector<int>& data, Metrics& m) {</code>
2	<code> int n = data.size();</code>
3	<code> auto start = Clock::now();</code>
4	<code> m.comparisons = 0;</code>
5	<code> m.swaps = 0;</code>
6	
7	<code> for (int i = 0; i < n; i++) {</code>
8	<code> for (int j = 0; j < n - i - 1; j++) {</code>
9	<code> if (data[j] > data[j+1]) {</code>
10	<code> swap(data[j], data[j+1]);</code>
11	<code> m.swaps++;</code>
12	<code> }</code>
13	<code> m.comparisons++;</code>
14	<code> }</code>
15	<code> }</code>
16	
17	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>
18	<code>}</code>

B. Output Program



```
>> Bubble Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.026600
Bubble Sort	Sorted	100	4950	0	0	0	0	0.008200
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.018100
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.010000
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.033700
Bubble Sort	Random	1000	499500	243190	0	0	0	1.231000
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.481800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	1.922200
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.556100
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	0.958600
Bubble Sort	Random	10000	49995000	25118148	0	0	0	103.403100
Bubble Sort	Sorted	10000	49995000	0	0	0	0	16.362000
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	122.017400
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	42.082200
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	90.979800

Gambar 1. Screenshot Output Bubble Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n)$, karena data sudah hampir terurut atau bahkan sudah terurut, sehingga tidak perlu melakukan banyak looping dan swap.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping berkali-kali dan swap berkali-kali.
- Worst case = $O(n^2)$, karena program jadi looping dan swap sebanyak mungkin. Contohnya pada reverse sorted, ia memiliki running time paling tinggi diantara data yang lain dengan jumlah yang sama.

2. Karakteristik algoritma

- Algoritma ini stabil, karna workflownya hanya swap-swap dengan elemen disampingnya. Tidak mengubah posisi elemen yang sudah benar.
- Algoritma ini termasuk in-place, karna swap dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah melakukan swap elemen bersebelahan jika $[j]$ lebih besar dari $[j+1]$. Datanya terurut mulai dari belakang, dari data paling besar hingga terkecil.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison.

- Jumlah swap tergantung pada bentuk data, pada best case, tentunya swapnya 0 karena data sudah sorted, tapi di worst case swapnya paling banyak dan sangat banyak.

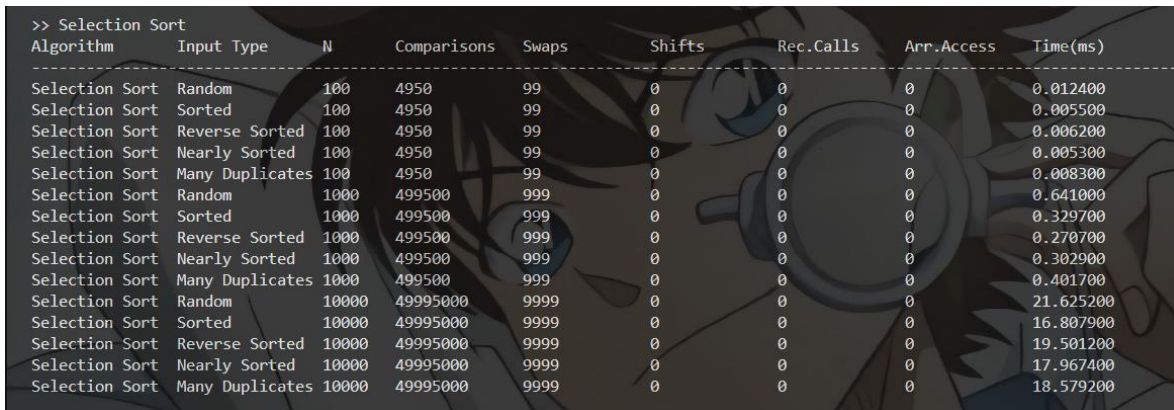
SELECTION SORT

A. Source Code

Tabel 2. Source Code Selection Sort

1	<code>void selection_sort(std::vector<int>& data, Metrics& m)</code>
	<code>{</code>
2	<code> int n = data.size();</code>
3	<code> auto start = Clock::now();</code>
4	<code> m.comparisons = 0;</code>
5	<code> m.swaps = 0;</code>
6	
7	<code> for (int i = 0; i < n-1; i++) {</code>
8	<code> int minVal = i;</code>
9	<code> for (int j = i+1; j < n; j++) {</code>
10	<code> if (data[j] < data[minVal]) {</code>
11	<code> minVal = j;</code>
12	<code> }</code>
13	<code> m.comparisons++;</code>
14	<code> }</code>
15	<code> swap(data[i], data[minVal]);</code>
16	<code> m.swaps++;</code>
17	<code> }</code>
18	
19	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>
20	<code>}</code>

B. Output Program



>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	99	0	0	0	0.012400
Selection Sort	Sorted	100	4950	99	0	0	0	0.005500
Selection Sort	Reverse Sorted	100	4950	99	0	0	0	0.006200
Selection Sort	Nearly Sorted	100	4950	99	0	0	0	0.005300
Selection Sort	Many Duplicates	100	4950	99	0	0	0	0.008300
Selection Sort	Random	1000	499500	999	0	0	0	0.641000
Selection Sort	Sorted	1000	499500	999	0	0	0	0.329700
Selection Sort	Reverse Sorted	1000	499500	999	0	0	0	0.270700
Selection Sort	Nearly Sorted	1000	499500	999	0	0	0	0.302900
Selection Sort	Many Duplicates	1000	499500	999	0	0	0	0.401700
Selection Sort	Random	10000	49995000	9999	0	0	0	21.625200
Selection Sort	Sorted	10000	49995000	9999	0	0	0	16.807900
Selection Sort	Reverse Sorted	10000	49995000	9999	0	0	0	19.501200
Selection Sort	Nearly Sorted	10000	49995000	9999	0	0	0	17.967400
Selection Sort	Many Duplicates	10000	49995000	9999	0	0	0	18.579200

Gambar 2. Screenshot Output Selection Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n^2)$, karena meskipun sudah sorted, algoritma tetap looping untuk mencari elemen terkecil.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping berkali-kali dan swap.
- Worst case = $O(n^2)$, karena program jadi looping dan swap sebanyak mungkin, ini sangat memakan waktu.

2. Karakteristik algoritma

- Algoritma ini tidak stabil, karna dengan swap elemen yang jauh, bisa menukar posisi relatif data duplikat dan bisa saja posisi elemen yang harusnya di depan jadi terkebelakang sementara karena swap tadi.
- Algoritma ini termasuk in-place, karna swap dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah program menelusuri array untuk mencari elemen paling kecil, lalu jika sudah ketemu, masuk variabel minVal, elemen ini akan di swap ke [i]. Sehingga pengurutan data terjadi dari depan.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison antara data[i] dengan minVal.

- Jumlah swap tergantung pada banyaknya data. Walau sorted, program menukar elemen dengan dirinya sendiri, karena `data[i]` di compare dengan variabel `minVal`.

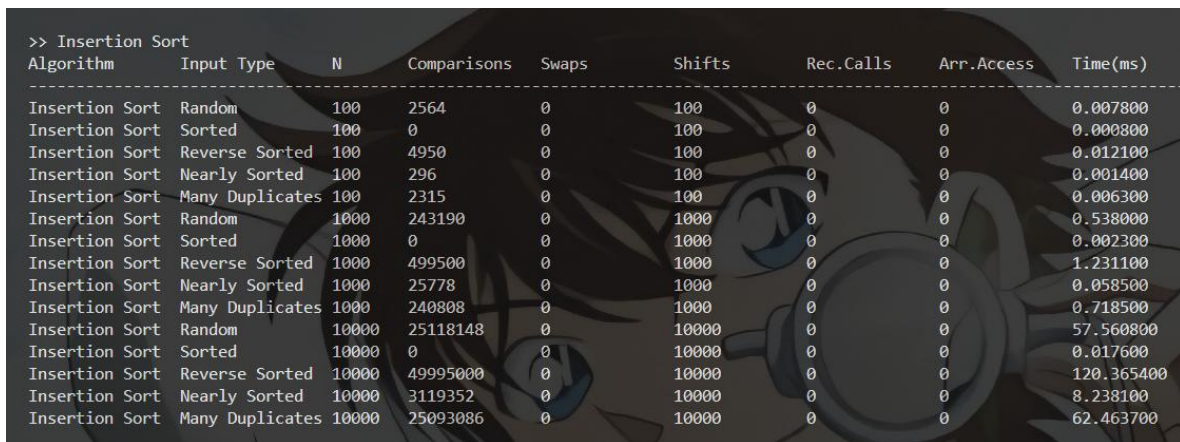
INSERTION SORT

A. Source Code

Tabel 3. Source Code Insertion Sort

1	<code>void insertion_sort(std::vector<int>& data, Metrics& m)</code>
	<code>{</code>
2	<code> int n = data.size();</code>
3	<code> auto start = Clock::now();</code>
4	<code> m.comparisons = 0;</code>
5	<code> m.shifts = 0;</code>
6	
7	<code> for (int i = 0; i < n; i++) {</code>
8	<code> int key = data[i];</code>
9	<code> int j = i - 1;</code>
10	<code> while (j >= 0 && data[j] > key) {</code>
11	<code> data[j+1] = data[j];</code>
12	<code> j--;</code>
13	<code> m.comparisons++;</code>
14	<code> }</code>
15	<code> data[j+1] = key;</code>
16	<code> m.shifts++;</code>
17	<code> }</code>
18	
19	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>
20	<code>}</code>

B. Output Program



>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2564	0	100	0	0	0.007800
Insertion Sort	Sorted	100	0	0	100	0	0	0.000800
Insertion Sort	Reverse Sorted	100	4950	0	100	0	0	0.012100
Insertion Sort	Nearly Sorted	100	296	0	100	0	0	0.001400
Insertion Sort	Many Duplicates	100	2315	0	100	0	0	0.006300
Insertion Sort	Random	1000	243190	0	1000	0	0	0.538000
Insertion Sort	Sorted	1000	0	0	1000	0	0	0.002300
Insertion Sort	Reverse Sorted	1000	499500	0	1000	0	0	1.231100
Insertion Sort	Nearly Sorted	1000	25778	0	1000	0	0	0.058500
Insertion Sort	Many Duplicates	1000	240808	0	1000	0	0	0.718500
Insertion Sort	Random	10000	25118148	0	10000	0	0	57.560800
Insertion Sort	Sorted	10000	0	0	10000	0	0	0.017600
Insertion Sort	Reverse Sorted	10000	49995000	0	10000	0	0	120.365400
Insertion Sort	Nearly Sorted	10000	3119352	0	10000	0	0	8.238100
Insertion Sort	Many Duplicates	10000	25093086	0	10000	0	0	62.463700

Gambar 3. Screenshot Output Insertion Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n)$, karena setiap elemen dibandingkan hanya sekali, lalu di insert ke posisi yang seharusnya.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping dan geser elemen untuk insert.
- Worst case = $O(n^2)$, karena program jadi looping sebanyak mungkin untuk compare dan shift.

2. Karakteristik algoritma

- Algoritma ini stabil, karna proses shift berhenti jika menemukan elemen dengan nilai yang sama atau posisi elemen sudah tepat.
- Algoritma ini termasuk in-place, karna shifts dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah mengambil nilai key di $[j+1]$, lalu dibandingkan dengan elemen di sebelah kirinya (j), jika lebih besar, maka geser nilainya ke kanan. Hingga key menemukan posisinya yang tepat, lalu nilai key ditaruh di indeks yang tepat itu.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison antara $data[j]$ dengan key.

- Jumlah shifts tergantung pada banyaknya data. Walau data sorted, tetap ada shift karena nilai data di indeks $j+1$ disimpan ke variabel key. Dan total shift sama dengan jumlah data karena nilai data dari indeks 0 sampai N disimpan ke key secara bergantian gitu.

MERGE SORT

A. Source Code

Tabel 4. Source Code Merge Sort

1	<code>void merge(std::vector<int>& data, int left, int mid,</code>
	<code>int right, Metrics& m) {</code>
2	<code> int n1 = mid - left + 1;</code>
3	<code> int n2 = right - mid;</code>
4	<code> m.comparisons = 0;</code>
5	
6	<code> std::vector<int> L(n1), R(n2);</code>
7	
8	<code> for (int i = 0; i < n1; i++) {</code>
9	<code> L[i] = data[left+i];</code>
10	<code> }</code>
11	<code> for (int j = 0; j < n2; j++) {</code>
12	<code> R[j] = data[mid + 1 + j];</code>
13	<code> }</code>
14	
15	<code> int i = 0, j = 0, k = left;</code>
16	
17	<code> while (i < n1 && j < n2) {</code>
18	<code> if (L[i] <= R[j]) {</code>
19	<code> data[k] = L[i];</code>
20	<code> i++;</code>
21	<code> } else {</code>
22	<code> data[k] = R[j];</code>
23	<code> j++;</code>
24	<code> }</code>
25	<code> k++;</code>
26	<code> m.comparisons++;</code>
27	<code> }</code>


```

28
29     while (i < n1) {
30         data[k] = L[i];
31         i++;
32         k++;
33     }
34
35     while (j < n2) {
36         data[k] = R[j];
37         j++;
38         k++;
39     }
40 }
41
42 void mergeSort(std::vector<int>& data, int left, int
right, Metrics& m) {
43     m.comparisons = 0;
44     m.recursive_calls = 0;
45     if (left >= right) {
46         m.comparisons++;
47         return;
48     }
49
50     int mid = left + (right - left) / 2;
51     mergeSort(data, left, mid, m);
52     m.recursive_calls++;
53     mergeSort(data, mid+1, right, m);
54     m.recursive_calls++;
55     merge(data, left, mid, right, m);
56 }
57

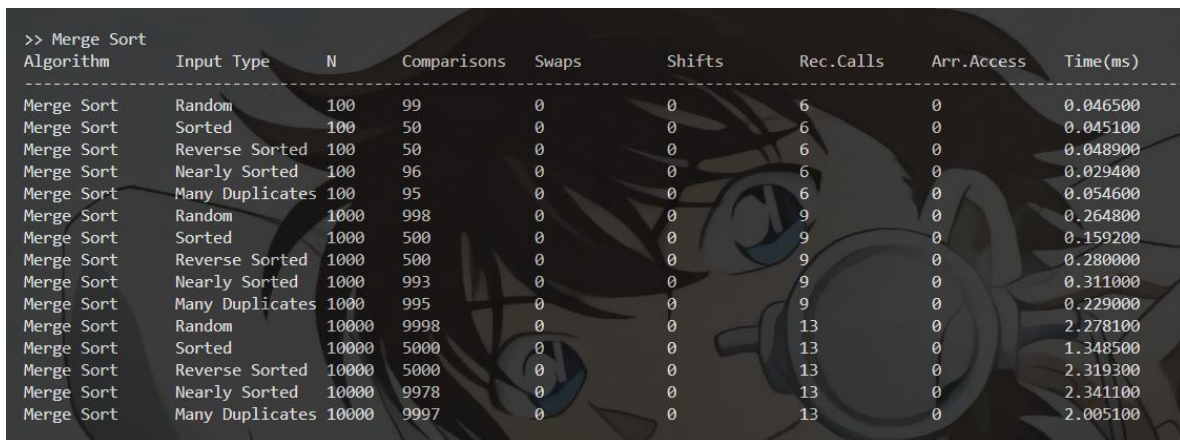
```

```

58 void merge_sort(std::vector<int>& data, Metrics& m) {
59     auto start = Clock::now();
60
61     mergeSort(data, 0, data.size()-1, m);
62
63     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
64 }

```

B. Output Program



Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	99	0	0	6	0	0.046500
Merge Sort	Sorted	100	50	0	0	6	0	0.045100
Merge Sort	Reverse Sorted	100	50	0	0	6	0	0.048900
Merge Sort	Nearly Sorted	100	96	0	0	6	0	0.029400
Merge Sort	Many Duplicates	100	95	0	0	6	0	0.054600
Merge Sort	Random	1000	998	0	0	9	0	0.264800
Merge Sort	Sorted	1000	500	0	0	9	0	0.159200
Merge Sort	Reverse Sorted	1000	500	0	0	9	0	0.280000
Merge Sort	Nearly Sorted	1000	993	0	0	9	0	0.311000
Merge Sort	Many Duplicates	1000	995	0	0	9	0	0.229000
Merge Sort	Random	10000	9998	0	0	13	0	2.278100
Merge Sort	Sorted	10000	5000	0	0	13	0	1.348500
Merge Sort	Reverse Sorted	10000	5000	0	0	13	0	2.319300
Merge Sort	Nearly Sorted	10000	9978	0	0	13	0	2.341100
Merge Sort	Many Duplicates	10000	9997	0	0	13	0	2.005100

Gambar 4. Screenshot Output Merge Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case, average case, worst case tetap $O(n \log n)$, karena array selalu dibagi jadi 2 bagian hingga terkecil.

2. Karakteristik algoritma

- Algoritma ini stabil, karna tidak memindah posisi relatif elemen, yang kiri tetaplah di kiri.
- Algoritma ini termasuk out-of-place, karna perlu array tambahan untuk menyimpan hasil penggabungan elemen yang dibagi 2 tadi.

- Mekanisme utamanya adalah array dibagi 2, lalu rekursif dan jadi terbagi 2 lagi hingga sisa 1 elemen. Lalu bagian-bagian tersebut digabungkan secara berurutan di array lain.

3. Analisis operasi

- Jumlah comparison tergantung pada bentuk dan banyaknya data, karna array selalu dibagi 2, jadi kalau datanya sorted, hanya pointer elemen kiri yang bergerak, membandingkan dengan elemen pertama di kanan. Kalau tidak sorted, pointer elemen kanan juga naik jika elemen sebelumnya dimasukkan ke array baru.
- Jumlah recursive calls tergantung pada banyaknya data, karena array selalu dibagi 2. Misalnya $N = 100$ selalu 6 recursive calls karna 6 kali membagi array menjadi 2 hingga tersisa 1 elemen di array.

QUICK SORT

A. Source Code

Tabel 5. Source Code Quick Sort

1	<code>int partition(std::vector<int>& data, Metrics& m, int</code>
	<code>low, int high) {</code>
2	<code> int pivot = data[high];</code>
3	<code> int i = low - 1;</code>
4	<code> m.swaps = 0;</code>
5	
6	<code> for (int j = low; j <= high - 1; j++) {</code>
7	<code> if (data[j] < pivot) {</code>
8	<code> i++;</code>
9	<code> swap(data[i], data[j]);</code>
10	<code> m.swaps++;</code>
11	<code> }</code>
12	<code> }</code>
13	
14	<code> swap(data[i + 1], data[high]);</code>
15	<code> m.swaps++;</code>
16	<code> return i + 1;</code>
17	<code>}</code>
18	
19	<code>void quickSort(std::vector<int>& data, Metrics& m, int</code>
	<code>low, int high) {</code>
20	<code> m.comparisons = 0;</code>
21	<code> if (low < high) {</code>
22	<code> int parti = partition(data, m, low, high);</code>
23	<code> m.comparisons++;</code>
24	
25	<code> quickSort(data, m, low, parti-1);</code>
26	<code> m.recursive_calls++;</code>

```

27         quickSort(data, m, parti+1, high);
28         m.recursive_calls++;
29     }
30 }
31
32 void quick_sort(std::vector<int>& data, Metrics& m) {
33     auto start = Clock::now();
34
35     quickSort(data, m, 0, data.size()-1);
36
37     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
38 }

```

B. Output Program

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	64	394	0	128	0	0.003700
Quick Sort	Sorted	100	99	5049	0	198	0	0.004100
Quick Sort	Reverse Sorted	100	99	2549	0	198	0	0.004300
Quick Sort	Nearly Sorted	100	95	2607	0	190	0	0.003200
Quick Sort	Many Duplicates	100	90	236	0	180	0	0.003000
Quick Sort	Random	1000	670	6125	0	1340	0	0.034800
Quick Sort	Sorted	1000	999	500499	0	1998	0	0.239300
Quick Sort	Reverse Sorted	1000	999	250499	0	1998	0	0.177600
Quick Sort	Nearly Sorted	1000	805	25081	0	1610	0	0.028300
Quick Sort	Many Duplicates	1000	900	4920	0	1800	0	0.031800
Quick Sort	Random	10000	6657	77996	0	13314	0	0.471100
Quick Sort	Sorted	10000	9999	50004999	0	19998	0	21.383900
Quick Sort	Reverse Sorted	10000	9999	25004999	0	19998	0	15.684500
Quick Sort	Nearly Sorted	10000	8255	458547	0	16510	0	0.404500
Quick Sort	Many Duplicates	10000	9001	67083	0	18002	0	0.405700

Gambar 5. Screenshot Output Quick Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n \log n)$, karna pivot yang dipilih untuk membagi array menjadi 2 sama besar.
- Average case = $O(n \log n)$, jika pemilihan pivot untuk membagi 2 arraynya tepat.

- Worst case = $O(n^2)$, jika salah pilih pivot, misalnya pivotnya di elemen pertama atau terakhir pada data sorted (ascending atau descending). Sehingga pembagian tidak seimbang.

2. Karakteristik algoritma

- Algoritma ini tidak stabil, karna pemindahan elemen sisi kiri dan kanan pivot yang berjauhan, memungkinkan memindah posisi relatif elemen yang nilainya sama.
- Algoritma ini termasuk in-place, karna partisi dilakukan di array yang sama (tapi perlu memori untuk call stack).
- Mekanisme utamanya adalah array dibagi 2, di kiri pivot dan kanan pivot. Lalu semua elemen yang lebih kecil dari pivot dipindah ke kiri, elemen lebih besar dari pivot dipindah ke kanan.

3. Analisis operasi

- Jumlah comparison tergantung pada partisi array, karna setiap elemen di kiri dan kanan pivot harus dibandingkan dengan nilai pivot, agar elemen mengetahui posisinya.
- Jumlah swap tergantung pada banyaknya elemen yang salah posisi, karna swap dilakukan jika elemen sisi kiri yang harusnya di kanan atau elemen kanan yang harusnya di kiri.
- Jumlah recursive calls tergantung pada banyaknya data, karena array selalu dibagi-bagi menjadi 2, dan juga tergantung pivotnya seimbang atau tidak.

RADIX SORT

A. Source Code

Tabel 6. Source Code Radix Sort

1	int getMax(std::vector<int>& data, int n) {
2	int max = data[0];
3	for (int i = 0; i < n; i++) {
4	if (data[i] > max) {
5	max = data[i];
6	}
7	}
8	return max;
9	}
10	
11	void count_sort(std::vector<int>& data, Metrics& m, int
	n, int exp) {
12	int output[n];
13	int i, count[10] = { 0 };
14	m.array_accesses = 0;
15	
16	for (i = 0; i < n; i++) {
17	count[(data[i] / exp) % 10]++;
18	}
19	
20	for (i = n-1; i >= 0; i--) {
21	output[count[(data[i] / exp) % 10] - 1] =
	data[i];
22	count[(data[i] / exp) % 10]--;
23	}
24	
25	for (i = 0; i < n; i++) {
26	data[i] = output[i];

```

27         m.array_accesses++;
28     }
29 }
30
31 void radixSort (std::vector<int>& data, Metrics& m, int
n) {
32     int maxVal = getMax(data, n);
33
34     for (int exp = 1; (maxVal / exp) > 0; exp *= 10) {
35         count_sort(data, m, n, exp);
36     }
37 }
38
39 void radix_sort(std::vector<int>& data, Metrics& m) {
40     if (data.empty()) return;
41     auto start = Clock::now();
42     int n = sizeof(data) / sizeof(data[0]);
43
44     radixSort(data, m, n);
45
46     m.time_ms      =      std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
47 }

```


B. Output Program



>> Radix Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	300	0.002500
Radix Sort	Sorted	100	0	0	0	0	300	0.001800
Radix Sort	Reverse Sorted	100	0	0	0	0	300	0.002300
Radix Sort	Nearly Sorted	100	0	0	0	0	300	0.002300
Radix Sort	Many Duplicates	100	0	0	0	0	200	0.002200
Radix Sort	Random	1000	0	0	0	0	4000	0.027800
Radix Sort	Sorted	1000	0	0	0	0	4000	0.013100
Radix Sort	Reverse Sorted	1000	0	0	0	0	4000	0.019100
Radix Sort	Nearly Sorted	1000	0	0	0	0	4000	0.013800
Radix Sort	Many Duplicates	1000	0	0	0	0	3000	0.010200
Radix Sort	Random	10000	0	0	0	0	50000	0.194500
Radix Sort	Sorted	10000	0	0	0	0	50000	0.167800
Radix Sort	Reverse Sorted	10000	0	0	0	0	50000	0.238600
Radix Sort	Nearly Sorted	10000	0	0	0	0	50000	0.161000
Radix Sort	Many Duplicates	10000	0	0	0	0	40000	0.118300

Gambar 6. Screenshot Output Radix Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case, average case, worst case tetap $O(d * (N + k))$, d adalah jumlah digit, N jumlah elemen, dan k adalah bucket representasi angka. Karena running time dipengaruhi oleh ketiga komponen tersebut.

2. Karakteristik algoritma

- Algoritma ini stabil, karna workflownya meletakkan nilai ke bucket sesuai digit.
- Algoritma ini termasuk out-of-place, karna adanya bucket, menyediakan tempat untuk menghitung elemen-elemen dengan nilai digit yang sama.
- Mekanisme utamanya adalah membuat bucket, 10 ruang. Lalu elemen dengan digit sekian, disimpan ke bucket indeks ke sekian. Dimulai dari satuan, puluhan, dan seterusnya.

3. Analisis operasi

- Jumlah array access bergantung pada banyaknya data dan jumlah digit angka terbesarnya. Karena sistemnya bukan comparison, swap, dan sebagainya, tapi memang sistem collecting elemen yang digitnya sesuai dengan indeks bucket yang disediakan. Semakin banyak data, semakin banyak array access.

ANALISIS PERFORMA

Perbandingan algoritma

1. Bubble sort, paling lambat, banyak comparison dan swap. Running time melonjak seiring bertambahnya data dan perbedaan bentuk data. Cocok untuk dataset kecil, tapi jarang digunakan.
2. Selection sort, lebih baik dari bubble sort, karna total swap berkurang, tapi lebih lambat dari insertion sort. Running time naik drastis seiring banyaknya data, beda dengan bubble sort. Cocok untuk dataset kecil, tapi mending insertion sort.
3. Insertion sort, lebih baik dari bubble dan selection, tapi kompleksitas masih belum efisien. Sangat cocok untuk dataset kecil.
4. Merge sort, algoritma stabil dan cepat. Walaupun mengakses memori yang lumayan. Cocok untuk dataset besar, terutama yang mengutamakan stabilitas.
5. Quick sort, algoritma paling cepat, namun tergantung pemilihan pivot yang baik. Sangat cocok untuk dataset besar, selain sangat cepat, juga hemat memori.
6. Radix sort, sangat cepat, namun memerlukan tambahan memori yang sangat besar. Cocok untuk dataset dengan tipe bilangan bulat saja.

Pengaruh distribusi data

1. Untuk sorted array, menjadikan bubble sort dan insertion sort sangat cepat. Namun menjadi worst case untuk quick sort.
2. Untuk nearly sorted, menjadikan bubble sort dan insertion sort menjadi efisien, cocok digunakan.
3. Untuk data random, bagus bagi quick sort dan merge sort.
4. Untuk data bilangan bulat berdigit, bagusnya pakai radix sort, paling bagus.

Hubungan teori kompleksitas dengan eksperimen

Hasil eksperimen menunjukkan pertumbuhan waktu pada masing-masing sorting dan semuanya sesuai. Ada yang naik drastis, ada yang stabil, ada yang lurus, dan sebagainya. Hasil eksperimen di dunia nyata (benchmark) mengkonfirmasi tren yang diprediksi oleh teori kompleksitas (Big-O).

TAUTAN GIT

Repository : <https://github.com/DSA26-ULM/module-4-sorting-soraawistaria>